

Movie Theatre Ticket Booking

P1 Database Design and P2

Executable MySQL schema, normalized entity design, transactional seat holds, idempotent payment-event handling, concurrency evidence, and the theatre/date showtime query.

Repository

<https://github.com/baigny/movie-seat-booking-theatre>

Verification: MySQL Community Server 8.4.9 | 20 checks passed | 1,000-request independent-seat benchmark at 64 workers

Movie theatre booking — P1 and P2

Repository: [baigny/movie-seat-booking-theatre](https://github.com/baigny/movie-seat-booking-theatre). The complete solution is on [feature/p1-p2-sql](https://github.com/baigny/movie-seat-booking-theatre/blob/main/feature/p1-p2-sql).

Executable MySQL schema, sample data, showtime query, and transaction tests. The schema follows **BCNF** and includes **concurrency verification**. Redis, queues, an HTTP backend, external payments, and automatic scheduling are outside the project's scope.

Files

- [P1](#): eleven BCNF base tables, constraints, indexes, sample data.
- [Schema design](#): every attribute, relationships, candidate keys, functional dependencies, and the integrity boundary.
- [Transaction API](#): atomic holds, confirmation, expiry, and the derived seat_availability view.
- [P2](#): theatre/date showtime query.
- [Test suite](#) and [execution report](#).
- [Migration](#): allocation-table conversion for compatible existing databases.

Installation

MySQL 8.0.16+ with InnoDB; tested on Community Server 8.4.9. Dates use Indian local time; money is INR. On a fresh project database, at the mysql prompt:

```
SET time_zone = '+05:30';
SOURCE C:/data-modelling-movie-theatre/sql/p1.sql;
SOURCE C:/data-modelling-movie-theatre/sql/booking_api.sql;
SOURCE C:/data-modelling-movie-theatre/sql/p2.sql;
```

Do not source P1 against populated tables. For a compatible existing database, back it up, stop application writes, run the migration once, then install booking_api.sql. Use mysql batch input that aborts on errors, without --force; DDL commits implicitly, so inspect a partially failed migration rather than rerunning blindly. Migration preflight rejects inconsistent allocations. Migration and routine installation were verified against the compatible sample schema; verification runs do not modify your existing database.

Applications must use the grants in [application_role.sql](#): SELECT plus EXECUTE on hold_seats, confirm_payment, and expire_booking only. Assign the role to a new application account with no other privileges. Do not grant table DML or the internal lock helper. Each public routine owns its transaction and must be called outside an existing transaction.

Sample rows

Table	Rows	Example
theatres	2	Starlight Cinema, Bengaluru; Moonlight Cinema, Chennai
screens	3	Two screens at theatre 1; one at theatre 2
movies	3	Journey to the Stars (120 min), The Last Train (105), Ocean of Dreams (135)
seats	8	Screen 1 A1/A2/B1/B2; screens 2 and 3 A1/A2

BOOKMYSHOW Theatre	TICKETING DATABASE	Rows	Example
shows	10		Fixed fixture dates September 28–October 4, 2026
users	2		Asha Rao, asha@example.com; Ravi Kumar, ravi@example.com
bookings	1		Booking 1: user 1, screen 1, September 28 at 10:00, confirmed, INR 400
show_seats	36		All valid positions for each fixture show
seat_allocations	2		Booking 1 owns A1/A2
booking_seats	2		Booking 1, A1/A2 at INR 200 each
payment_events	1		demo_provider / evt_demo_001, INR 400, applied

Full example values are in P1. The availability view derives two sold seats and 34 available seats. The historical booking was created September 27 at 18:00, held until 18:10, and paid at 18:05 (processed 18:05:01). Fixture dates stay fixed; tests create future shows relative to the server clock.

Transaction behavior

`hold_seats(user_id, show_id, reference, seats_json, ttl_seconds)` takes 1–20 distinct positions and a 1–900 second hold. JSON example: `[{"row": "B", "number": 1}, {"row": "B", "number": 2}]`. Inventory locks are taken in row/number order. Any unavailable/invalid seat rolls back the new booking, all allocations, and all items. Success returns `booking_id`. Expired allocations remain protected until cleanup releases them; a competing hold cannot steal them.

`expire_booking(booking_id)` locks the booking and inventory, releasing only that booking's allocations when a held deadline has passed. Confirmed bookings are never expired. An external worker can call it; no scheduler is installed.

`confirm_payment(booking_id, provider, event_id, amount)` serializes with expiry on the booking lock, checks totals and complete ownership, and confirms once. Duplicate events return the existing outcome; altered replay payloads are rejected. Late success or a distinct extra payment returns `refund_required` without reclaiming reassigned seats. No external refund is executed.

The clock is evaluated after lock waits using `SYSDATE()`, rather than `NOW()`'s statement-start time. Do not enable `sysdate-is-now`; use row-based logging if replicating these routines. Each public routine uses `READ COMMITTED`. Existing bookings lock before inventory; inventory uses canonical order. Every SQL exception rolls back the transaction. Clients retry only 1205 and 1213 with bounded backoff/jitter, restarting the entire call. Business rejections are not retried. The suite induces and verifies both a timeout and a real deadlock.

MySQL references: [locking reads](#) and [JSON_TABLE](#).

The API chooses pessimistic seat-row locks for contested inventory. The trade-off with an optimistic claim design, including the unique-claim/version state that would be needed with this normalized schema, is documented in [SCHEMA DESIGN.md](#).

P2 query

P2 uses theatre 1 and September 28, 2026 by default. The WHERE clause uses a half-open date range without a function on `starts_at`. Order is movie title, start time, then show ID.

Movie	Screen	Time
Journey to the Stars	Screen 1	10:00:00
Journey to the Stars	Screen 1	14:00:00
The Last Train	Screen 2	11:00:00

Theatre 2 that day returns Ocean of Dreams at 10:00. Theatre 1 on September 29 returns The Last Train at 10:00; October 5 has no rows. The intended date picker offers today plus six days in Indian local time; no frontend is included. Edit the SET inputs for another selection; sourcing P2 resets its defaults.

Verified concurrency evidence

The [report](#) records a successful MySQL 8.4.9 run:

- 1,000 competing two-seat requests at 64 workers: one winner, 999 rejected in 20.078 seconds; no partial losing bookings or allocations.
- Independent-seat workload: 1,000 of 1,000 holds succeeded at 64 workers in 20.806 seconds (48.06 requests/second). Latency was p50 1,193 ms, p95 1,871 ms, p99 2,295 ms, and max 4,117 ms.
- 100 concurrent duplicate confirmations: one applied event and stable ownership.
- 30 overlapping expiry/payment races: 15 expired and 15 live holds; stale confirmation cannot take seats from a replacement booking.
- Expiry while waiting for a lock, timeout rollback/retry, and an actual deadlock victim followed by whole-transaction retry all passed.
- Nine negative constraint cases, invalid/duplicate seat requests, all four P2 cases, migration preservation, and restricted-client access passed.

All 20 checks in the execution report passed.

The suite asserts no duplicate show-seat ownership, no invalid inventory, no partial active bookings, no lost allocations, and no duplicate applied payment effects. Lock waits are observed in performance_schema. The benchmark used 64 worker threads, not 1,000 simultaneous connections. It launches one MySQL CLI process and TCP connection per request, so measurements include local process and connection startup overhead. Treat these figures as a reproducible local baseline, not production capacity or a universal comparison with an unimplemented optimistic strategy.

Reproduce

Use Python 3.12 and mysql CLI against an isolated test server:

```
python sql/tests/verify_bcnf.py --port 13307 --attempts 1000 --workers 64 --benchmark-requests 1000
```

Only the Python standard library is needed. Use --login-path for saved local credentials. The suite creates GUID-named schemas and leaves them for inspection; it never drops or overwrites an application database. Administrative test access is required to create routines, observe locks, and create/remove a temporary restricted account. Passwords are not stored in this repository. The migration test uses the a2153ef compatibility tag included in the repository.

The integration suite and its JSON report provide the execution evidence.

BCNF schema design

The [P1 schema](#) contains eleven base tables. Valid show inventory and current ownership are separate relations. No base table repeats the show determined by an allocation's booking.

Attributes and constraints

All columns are NOT NULL unless marked nullable. Integer IDs and seat numbers are UNSIGNED. All tables use InnoDB; foreign keys use RESTRICT on update/delete. Money is INR. Text defaults to utf8mb4_0900_ai_ci; payment identifiers use ascii_bin.

Table	Attributes and types	Keys / rules
theatres	theatre_id INT; theatre_name VARCHAR(150); address VARCHAR(500)	Auto-increment PK theatre_id
screens	screen_id INT; theatre_id INT; screen_name VARCHAR(50)	Auto-increment PK screen_id; unique theatre/name; theatre FK
movies	movie_id INT; title VARCHAR(200); duration_minutes SMALLINT	Auto-increment PK movie_id; positive duration
seats	seat_id INT; screen_id INT; row_label VARCHAR(5); seat_number SMALLINT	Auto-increment PK seat_id; unique screen/row/number; screen FK; positive number
shows	show_id INT; movie_id INT; screen_id INT; starts_at DATETIME; ticket_price DECIMAL(10,2)	Auto-increment PK show_id; unique screen/start; movie/screen FKs; nonnegative price
users	user_id INT; full_name VARCHAR(150); email VARCHAR(254)	Auto-increment PK user_id; unique email
bookings	booking_id INT; user_id INT; screen_id INT; starts_at DATETIME; booking_reference CHAR(36); status ENUM; created_at DATETIME; hold_expires_at DATETIME nullable; total_amount DECIMAL(10,2)	Auto-increment PK booking_id; unique reference; user FK; screen/start FK to shows; nonnegative amount; held requires deadline
show_seats	screen_id INT; starts_at DATETIME; row_label VARCHAR(5); seat_number SMALLINT	All four columns form PK; show FK on screen/start; physical-seat FK on screen/row/number
seat_allocations	booking_id INT; row_label VARCHAR(5); seat_number SMALLINT	All three columns form PK; booking FK; positive number
booking_seats	booking_id INT; row_label VARCHAR(5); seat_number SMALLINT; purchase_price DECIMAL(10,2)	PK booking/row/number; booking FK; positive number; nonnegative historical price
payment_events	payment_event_id BIGINT; provider VARCHAR(50); provider_event_id VARCHAR(191); booking_id INT; event_type ENUM; amount DECIMAL(10,2); processing_status ENUM; received_at DATETIME; processed_at DATETIME nullable	Auto-increment PK event ID; unique provider/event; booking FK; nonnegative amount; pending iff processed_at is NULL

Booking statuses: held (default), confirmed, expired, cancelled, payment_failed. created_at defaults to CURRENT_TIMESTAMP. Event types: payment_succeeded, payment_failed, refund_succeeded. Processing states: pending (default), applied, ignored, refund_required. received_at defaults to CURRENT_TIMESTAMP.

Relationships and indexes

A theatre has many screens; a screen has many physical seats and shows; each show has one movie. Bookings reference a customer and the show's unique (screen_id, starts_at) key. Inventory lists valid seats per show. Allocations derive their show through bookings. Historical booking items survive expiry while current allocations are released. A booking can have many payment events.

Foreign keys and unique keys have supporting indexes. bookings also indexes (user_id, created_at); payment_events indexes (booking_id, received_at). The inventory primary key is the seat mutex. The shows screen/start key supports P2. Allocation checks join the show's bookings to their allocations through booking-leading keys. This normalized join has a cost; the report measures correctness locally rather than claiming production throughput.

Functional dependencies and normal forms

Addresses, names, prices, dates, and statuses are atomic display/business values (1NF). Names/titles/addresses are not assumed unique. Prices do not depend on movie alone. Email equality follows its collation. Historical item prices may differ within a booking, for example through discounts.

Relation	Candidate keys	Nontrivial dependencies
theatres	theatre_id	ID determines name/address
screens	screen_id; theatre_id/screen_name	Either key determines all attributes
movies	movie_id	ID determines title/duration
seats	seat_id; screen_id/row_label/seat_number	Either key determines all attributes
shows	show_id; screen_id/starts_at	Either key determines movie/price and other key columns
users	user_id; email	Either key determines all attributes
bookings	booking_id; booking_reference	Either key determines user, show, lifecycle facts and total
show_seats	screen_id/starts_at/row_label/seat_number	All-key relation; no additional nontrivial dependencies
seat_allocations	booking_id/row_label/seat_number	All-key relation; no additional nontrivial dependencies
booking_seats	booking_id/row_label/seat_number	Complete key determines historical purchase price
payment_events	payment_event_id; provider/provider_event_id	Either key determines all event facts

Every nontrivial determinant is a candidate key or superkey under these stated dependencies. Thus each base relation satisfies BCNF, and consequently 3NF and 2NF, in addition to the atomic-value 1NF assumptions. Nullable timestamps represent absent lifecycle values; they do not introduce a non-key determinant.

seat_allocations stores booking and seat identifiers. Show coordinates exist in bookings, where booking_id is a key. Ownership status is derived from bookings instead of being stored per seat. seat_availability is a joined view, not a base table, and does not reintroduce stored redundancy.

Locking trade-offs

The transaction API uses pessimistic InnoDB row locks on the show_seats row for each requested seat. The inventory

row is the row, competing transactions for the same seat wait, then re-check committed allocations before inserting. This keeps the existing normalized ownership model and makes a multi-seat hold all-or-nothing. Its cost is lock waiting on hot seats, deadlock handling, and reduced throughput when many requests target the same inventory. The API limits each hold to 20 seats, locks them in canonical order, and retries only deadlock or lock-timeout errors at the client boundary.

An optimistic design would first read availability without locking and attempt an atomic conditional claim, using a version check or a unique key for the show-seat. Uncontended requests can avoid waiting for a read lock, but competing requests do extra work and losers must retry or fail after a write conflict. This schema deliberately derives the show through bookings, so `seat_allocations(booking_id, row_label, seat_number)` alone cannot enforce a unique show-seat claim across different bookings. An optimistic implementation would therefore need a dedicated claim relation keyed by show and seat, or versioned claim state on `show_seats`, plus transactional multi-seat rollback. Adding that state would change the present schema and integrity boundary. Given the expected hot-seat contention, explicit pessimistic locking is the simpler choice; the benchmark reports both contention correctness and uncontended-seat throughput instead of assuming one strategy is universally faster.

Integrity boundary

Unique ownership across different bookings is enforced by the transaction API under an inventory-row lock, rather than by duplicating show coordinates in the allocation key. The API also validates inventory membership, complete item ownership, amount equality, and expiry. All failed holds roll back the entire request. Applications must have only `SELECT` and `EXECUTE` on the public routines, with no direct table DML and no access to the internal lock helper. The suite tests this privilege boundary. Administrators can bypass business rules through direct DML and are outside the client guarantee.

Show schedules/layouts must remain stable while sales operate. Administrative rescheduling, external refunds, scheduling a cleanup worker, Redis, and queues are outside the project's scope. Concurrency correctness is documented with the actual test counts and limits in [README](#), not inferred from BCNF.

P1: executable schema and sample data

```
-- P1: MySQL schema and sample data.
-- Target: MySQL 8.0.16+ with InnoDB.
-- Batch 1: create and select the project database.
-- Inventory and current allocations are separate BCNF relations.

CREATE DATABASE IF NOT EXISTS movie_seat_booking
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_0900_ai_ci;

USE movie_seat_booking;

-- Batch 2: theatres and their screens.
CREATE TABLE theatres (
  theatre_id INT UNSIGNED NOT NULL AUTO_INCREMENT,
  theatre_name VARCHAR(150) NOT NULL,
  address VARCHAR(500) NOT NULL,
  PRIMARY KEY (theatre_id)
) ENGINE=InnoDB;

CREATE TABLE screens (
  screen_id INT UNSIGNED NOT NULL AUTO_INCREMENT,
  theatre_id INT UNSIGNED NOT NULL,
  screen_name VARCHAR(50) NOT NULL,
  PRIMARY KEY (screen_id),
  CONSTRAINT uq_screens_theatre_name UNIQUE (theatre_id, screen_name),
  CONSTRAINT fk_screens_theatre FOREIGN KEY (theatre_id)
    REFERENCES theatres (theatre_id)
    ON DELETE RESTRICT ON UPDATE RESTRICT
) ENGINE=InnoDB;

-- Batch 3: fictional sample theatres and screens (run once).
INSERT INTO theatres (theatre_id, theatre_name, address) VALUES
  (1, 'Starlight Cinema', '10 Sample Road, Bengaluru, Karnataka, India'),
  (2, 'Moonlight Cinema', '20 Example Road, Chennai, Tamil Nadu, India');

INSERT INTO screens (screen_id, theatre_id, screen_name) VALUES
  (1, 1, 'Screen 1'),
  (2, 1, 'Screen 2'),
  (3, 2, 'Screen 1');

-- Batch 4: movie catalogue and physical seats.
CREATE TABLE movies (
  movie_id INT UNSIGNED NOT NULL AUTO_INCREMENT,
  title VARCHAR(200) NOT NULL,
  duration_minutes SMALLINT UNSIGNED NOT NULL,
  PRIMARY KEY (movie_id),
  CONSTRAINT chk_movies_duration CHECK (duration_minutes > 0)
) ENGINE=InnoDB;

CREATE TABLE seats (
  seat_id INT UNSIGNED NOT NULL AUTO_INCREMENT,
  screen_id INT UNSIGNED NOT NULL,
  row_label VARCHAR(5) NOT NULL,
  seat_number SMALLINT UNSIGNED NOT NULL,
  PRIMARY KEY (seat_id),
  CONSTRAINT uq_seats_position UNIQUE (screen_id, row_label, seat_number),
  CONSTRAINT chk_seats_number CHECK (seat_number > 0),
  CONSTRAINT fk_seats_screen FOREIGN KEY (screen_id)
    REFERENCES screens (screen_id)
    ON DELETE RESTRICT ON UPDATE RESTRICT
) ENGINE=InnoDB;

-- Batch 5: fictional movies and a small demonstration seating layout.
INSERT INTO movies (movie_id, title, duration_minutes) VALUES
  (1, 'Journey to the Stars', 120),
  (2, 'The Last Train', 105),
  (3, 'Ocean of Dreams', 135);

INSERT INTO seats (seat_id, screen_id, row_label, seat_number) VALUES
  (1, 1, 'A', 1),
  (2, 1, 'A', 2),
  (3, 1, 'B', 1),
  (4, 1, 'B', 2),
  (5, 2, 'A', 1),
  (6, 2, 'A', 2),
  (7, 3, 'A', 1),
  (8, 3, 'A', 2);

-- Batch 6: scheduled shows and booking customers.
```


BOOKING API: THEATRE TICKETING DATABASE

```
CREATE TABLE shows (
  show_id INT UNSIGNED NOT NULL AUTO_INCREMENT,
  movie_id INT UNSIGNED NOT NULL,
  screen_id INT UNSIGNED NOT NULL,
  starts_at DATETIME NOT NULL,
  ticket_price DECIMAL(10,2) NOT NULL,
  PRIMARY KEY (show_id),
  CONSTRAINT uq_shows_screen_start UNIQUE (screen_id, starts_at),
  CONSTRAINT chk_shows_price CHECK (ticket_price >= 0),
  CONSTRAINT fk_shows_movie FOREIGN KEY (movie_id)
    REFERENCES movies (movie_id)
    ON DELETE RESTRICT ON UPDATE RESTRICT,
  CONSTRAINT fk_shows_screen FOREIGN KEY (screen_id)
    REFERENCES screens (screen_id)
    ON DELETE RESTRICT ON UPDATE RESTRICT
) ENGINE=InnoDB;

CREATE TABLE users (
  user_id INT UNSIGNED NOT NULL AUTO_INCREMENT,
  full_name VARCHAR(150) NOT NULL,
  email VARCHAR(254) NOT NULL,
  PRIMARY KEY (user_id),
  CONSTRAINT uq_users_email UNIQUE (email)
) ENGINE=InnoDB;

-- Batch 7: fixed-date sample shows and fictional customers (run once).
INSERT INTO shows (show_id, movie_id, screen_id, starts_at, ticket_price) VALUES
  (1, 1, 1, '2026-09-28 10:00:00', 200.00),
  (2, 1, 1, '2026-09-28 14:00:00', 220.00),
  (3, 2, 2, '2026-09-28 11:00:00', 180.00),
  (4, 3, 3, '2026-09-28 10:00:00', 250.00),
  (5, 2, 1, '2026-09-29 10:00:00', 200.00),
  (6, 3, 1, '2026-09-30 10:00:00', 240.00),
  (7, 1, 1, '2026-10-01 10:00:00', 200.00),
  (8, 2, 1, '2026-10-02 10:00:00', 200.00),
  (9, 3, 1, '2026-10-03 10:00:00', 260.00),
  (10, 1, 1, '2026-10-04 10:00:00', 220.00);

INSERT INTO users (user_id, full_name, email) VALUES
  (1, 'Asha Rao', 'asha@example.com'),
  (2, 'Ravi Kumar', 'ravi@example.com');

-- Batch 8: bookings and per-show seat inventory.
CREATE TABLE bookings (
  booking_id INT UNSIGNED NOT NULL AUTO_INCREMENT,
  user_id INT UNSIGNED NOT NULL,
  screen_id INT UNSIGNED NOT NULL,
  starts_at DATETIME NOT NULL,
  booking_reference CHAR(36) NOT NULL,
  status ENUM('held', 'confirmed', 'expired', 'cancelled', 'payment_failed')
    NOT NULL DEFAULT 'held',
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  hold_expires_at DATETIME NULL,
  total_amount DECIMAL(10,2) NOT NULL,
  PRIMARY KEY (booking_id),
  CONSTRAINT uq_bookings_reference UNIQUE (booking_reference),
  KEY ix_bookings_user_created (user_id, created_at),
  CONSTRAINT chk_bookings_amount CHECK (total_amount >= 0),
  CONSTRAINT chk_bookings_hold_expiry CHECK (
    status <> 'held' OR hold_expires_at IS NOT NULL
  ),
  CONSTRAINT fk_bookings_user FOREIGN KEY (user_id)
    REFERENCES users (user_id)
    ON DELETE RESTRICT ON UPDATE RESTRICT,
  CONSTRAINT fk_bookings_show FOREIGN KEY (screen_id, starts_at)
    REFERENCES shows (screen_id, starts_at)
    ON DELETE RESTRICT ON UPDATE RESTRICT
) ENGINE=InnoDB;

CREATE TABLE show_seats (
  screen_id INT UNSIGNED NOT NULL,
  starts_at DATETIME NOT NULL,
  row_label VARCHAR(5) NOT NULL,
  seat_number SMALLINT UNSIGNED NOT NULL,
  PRIMARY KEY (screen_id, starts_at, row_label, seat_number),
  CONSTRAINT fk_show_seats_show FOREIGN KEY (screen_id, starts_at)
    REFERENCES shows (screen_id, starts_at)
    ON DELETE RESTRICT ON UPDATE RESTRICT,
  CONSTRAINT fk_show_seats_seat FOREIGN KEY (screen_id, row_label, seat_number)
    REFERENCES seats (screen_id, row_label, seat_number)
    ON DELETE RESTRICT ON UPDATE RESTRICT
) ENGINE=InnoDB;

-- A booking determines its show. Do not repeat that show in allocations.
-- Only the transactional routines may write this table (see booking_api.sql).
CREATE TABLE seat_allocations (
  booking_id INT UNSIGNED NOT NULL,
  row_label VARCHAR(5) NOT NULL,
  seat_number SMALLINT UNSIGNED NOT NULL,
  PRIMARY KEY (booking_id, row_label, seat_number),
  CONSTRAINT chk_allocations_number CHECK (seat_number > 0),
  CONSTRAINT fk_allocations_booking FOREIGN KEY (booking_id)
    REFERENCES bookings (booking_id)
    ON DELETE RESTRICT ON UPDATE RESTRICT
) ENGINE=InnoDB;
```

Batch 9: historical booking items and payment event processing.

BOOKING SHOW THEATRE TICKETING DATABASE

```
CREATE TABLE booking_seats (
  booking_id INT UNSIGNED NOT NULL,
  row_label VARCHAR(5) NOT NULL,
  seat_number SMALLINT UNSIGNED NOT NULL,
  purchase_price DECIMAL(10,2) NOT NULL,
  PRIMARY KEY (booking_id, row_label, seat_number),
  CONSTRAINT chk_booking_seats_number CHECK (seat_number > 0),
  CONSTRAINT chk_booking_seats_price CHECK (purchase_price >= 0),
  CONSTRAINT fk_booking_seats_booking FOREIGN KEY (booking_id)
    REFERENCES bookings (booking_id)
    ON DELETE RESTRICT ON UPDATE RESTRICT
) ENGINE=InnoDB;

CREATE TABLE payment_events (
  payment_event_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  provider VARCHAR(50) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
  provider_event_id VARCHAR(191) CHARACTER SET ascii COLLATE ascii_bin NOT NULL,
  booking_id INT UNSIGNED NOT NULL,
  event_type ENUM('payment_succeeded', 'payment_failed', 'refund_succeeded')
    NOT NULL,
  amount DECIMAL(10,2) NOT NULL,
  processing_status ENUM('pending', 'applied', 'ignored', 'refund_required')
    NOT NULL DEFAULT 'pending',
  received_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  processed_at DATETIME NULL,
  PRIMARY KEY (payment_event_id),
  CONSTRAINT uq_payment_events_provider_event UNIQUE (provider, provider_event_id),
  KEY ix_payment_events_booking (booking_id, received_at),
  CONSTRAINT chk_payment_events_amount CHECK (amount >= 0),
  CONSTRAINT chk_payment_events_processed CHECK (
    (processing_status = 'pending' AND processed_at IS NULL)
    OR (processing_status <> 'pending' AND processed_at IS NOT NULL)
  ),
  CONSTRAINT fk_payment_events_booking FOREIGN KEY (booking_id)
    REFERENCES bookings (booking_id)
    ON DELETE RESTRICT ON UPDATE RESTRICT
) ENGINE=InnoDB;

-- Batch 10: fixed historical booking and inventory for all sample shows (run once).
-- Batch 11 will supply the booking's two items and successful payment event.
INSERT INTO bookings (
  booking_id, user_id, screen_id, starts_at, booking_reference,
  status, created_at, hold_expires_at, total_amount
) VALUES (
  1, 1, 1, '2026-09-28 10:00:00', '00000000-0000-4000-8000-000000000001',
  'confirmed', '2026-09-27 18:00:00', '2026-09-27 18:10:00', 400.00
);

INSERT INTO show_seats (
  screen_id, starts_at, row_label, seat_number
)
SELECT sh.screen_id, sh.starts_at, st.row_label, st.seat_number
FROM shows AS sh
JOIN seats AS st ON st.screen_id = sh.screen_id;

INSERT INTO seat_allocations (booking_id, row_label, seat_number)
VALUES (1, 'A', 1), (1, 'A', 2);

-- Batch 11: historical items and payment for booking 1 (run once).
INSERT INTO booking_seats (booking_id, row_label, seat_number, purchase_price)
VALUES
  (1, 'A', 1, 200.00),
  (1, 'A', 2, 200.00);

INSERT INTO payment_events (
  payment_event_id, provider, provider_event_id, booking_id, event_type,
  amount, processing_status, received_at, processed_at
) VALUES (
  1, 'demo_provider', 'evt_demo_001', 1, 'payment_succeeded',
  400.00, 'applied', '2026-09-27 18:05:00', '2026-09-27 18:05:01'
);
```

Transactional seat booking API

```
-- Install after P1, or after migration 001. Top-level calls own transactions.
-- Application accounts get EXECUTE on hold_seats, confirm_payment, expire_booking
-- and SELECT only. No direct DML and no EXECUTE on the internal lock helper.
USE movie_seat_booking;
CREATE VIEW seat_availability AS
SELECT s.screen_id, s.starts_at, s.row_label, s.seat_number, ownership.booking_id,
CASE WHEN ownership.booking_id IS NULL THEN 'available'
WHEN ownership.booking_status = 'confirmed' THEN 'sold' ELSE 'held' END AS status,
ownership.hold_expires_at
FROM show_seats s
LEFT JOIN (
SELECT a.booking_id, a.row_label, a.seat_number, b.screen_id, b.starts_at,
b.status AS booking_status, b.hold_expires_at
FROM seat_allocations a JOIN bookings b ON b.booking_id = a.booking_id
) ownership ON ownership.screen_id = s.screen_id AND ownership.starts_at = s.starts_at
AND ownership.row_label = s.row_label AND ownership.seat_number = s.seat_number;
DELIMITER //
CREATE PROCEDURE lock_booking_inventory(IN p_booking INT UNSIGNED)
SQL SECURITY DEFINER
BEGIN
DECLARE done BOOLEAN DEFAULT FALSE;
DECLARE v_screen INT UNSIGNED;
DECLARE v_start DATETIME;
DECLARE v_row VARCHAR(5);
DECLARE v_num SMALLINT UNSIGNED;
DECLARE found_seat SMALLINT UNSIGNED;
DECLARE items CURSOR FOR
SELECT b.screen_id, b.starts_at, bs.row_label, bs.seat_number
FROM bookings b JOIN booking_seats bs ON bs.booking_id = b.booking_id
WHERE b.booking_id = p_booking ORDER BY bs.row_label, bs.seat_number;
DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;
OPEN items;
item_loop: LOOP
FETCH items INTO v_screen, v_start, v_row, v_num;
IF done THEN LEAVE item_loop; END IF;
SET found_seat = NULL;
SELECT seat_number INTO found_seat FROM show_seats
WHERE screen_id = v_screen AND starts_at = v_start
AND row_label = v_row AND seat_number = v_num FOR UPDATE;
IF found_seat IS NULL THEN
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'missing inventory';
END IF;
END LOOP;
CLOSE items;
END//

CREATE PROCEDURE hold_seats(
IN p_user INT UNSIGNED, IN p_show INT UNSIGNED, IN p_reference CHAR(36),
IN p_seats JSON, IN p_ttl_seconds INT)
SQL SECURITY DEFINER
BEGIN
DECLARE done BOOLEAN DEFAULT FALSE;
DECLARE v_screen INT UNSIGNED;
DECLARE v_start DATETIME;
DECLARE v_price DECIMAL(10,2);
DECLARE v_booking INT UNSIGNED;
DECLARE v_row VARCHAR(5);
DECLARE v_num SMALLINT UNSIGNED;
DECLARE v_found SMALLINT UNSIGNED;
DECLARE v_count INT;
DECLARE v_unique INT;
DECLARE v_now DATETIME;
DECLARE requested CURSOR FOR
SELECT jt.row_label, jt.seat_number FROM JSON_TABLE(p_seats, '$[*]'
COLUMNS(row_label VARCHAR(5) PATH '$.row' ERROR ON EMPTY ERROR ON ERROR,
seat_number INT PATH '$.number' ERROR ON EMPTY ERROR ON ERROR)) jt
ORDER BY jt.row_label, jt.seat_number;
DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;
DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;
IF p_seats IS NULL OR JSON_TYPE(p_seats) <> 'ARRAY' OR JSON_LENGTH(p_seats) NOT BETWEEN 1 AND 20
OR p_ttl_seconds IS NULL OR p_ttl_seconds NOT BETWEEN 1 AND 900 THEN
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'invalid seats or hold duration';
END IF;
SELECT COUNT(*), COUNT(DISTINCT jt.row_label, jt.seat_number)
INTO v_count, v_unique FROM JSON_TABLE(p_seats, '$[*]'
COLUMNS(row_label VARCHAR(5) PATH '$.row' ERROR ON EMPTY ERROR ON ERROR,
seat_number INT PATH '$.number' ERROR ON EMPTY ERROR ON ERROR)) jt;
IF v_count <> v_unique THEN
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'duplicate or null requested seat';
```

BOOKING SHOW | THEATRE TICKETING DATABASE

```

END IF;
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
START TRANSACTION;
SELECT screen_id, starts_at, ticket_price INTO v_screen, v_start, v_price
FROM shows WHERE show_id = p_show;
IF v_screen IS NULL OR v_start <= SYSDATE() THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'unknown or started show';
END IF;
INSERT INTO bookings (user_id, screen_id, starts_at, booking_reference,
    status, hold_expires_at, total_amount)
VALUES (p_user, v_screen, v_start, p_reference, 'held',
    DATE_ADD(SYSDATE(), INTERVAL p_ttl_seconds SECOND), v_price * v_count);
SET v_booking = LAST_INSERT_ID();
SET done = FALSE;
OPEN requested;
seat_loop: LOOP
    FETCH requested INTO v_row, v_num;
    IF done THEN LEAVE seat_loop; END IF;
    SET v_found = NULL;
    SELECT seat_number INTO v_found FROM show_seats
    WHERE screen_id = v_screen AND starts_at = v_start
    AND row_label = v_row AND seat_number = v_num FOR UPDATE;
    IF v_found IS NULL THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'seat does not belong to show';
    END IF;
    -- Inventory mutex is held; RC sees any winner committed while waiting.
    IF EXISTS (SELECT 1 FROM seat_allocations a JOIN bookings b
        ON b.booking_id = a.booking_id WHERE b.screen_id = v_screen
        AND b.starts_at = v_start AND a.row_label = v_row AND a.seat_number = v_num) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'seat unavailable';
    END IF;
    INSERT INTO seat_allocations VALUES (v_booking, v_row, v_num);
    INSERT INTO booking_seats VALUES (v_booking, v_row, v_num, v_price);
END LOOP;
CLOSE requested;
SET v_now = SYSDATE();
IF v_start <= v_now THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'show started during wait';
END IF;
UPDATE bookings SET created_at = v_now,
    hold_expires_at = DATE_ADD(v_now, INTERVAL p_ttl_seconds SECOND)
WHERE booking_id = v_booking;
COMMIT;
SELECT v_booking AS booking_id, 'held' AS outcome;
END//

CREATE PROCEDURE expire_booking(IN p_booking INT UNSIGNED)
SQL SECURITY DEFINER
BEGIN
    DECLARE v_status VARCHAR(20);
    DECLARE v_expiry DATETIME;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;
    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
    START TRANSACTION;
    SELECT status, hold_expires_at INTO v_status, v_expiry
    FROM bookings WHERE booking_id = p_booking FOR UPDATE;
    IF v_status IS NULL THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'unknown booking'; END IF;
    CALL lock_booking_inventory(p_booking);
    IF v_status = 'held' AND v_expiry <= SYSDATE() THEN
        DELETE FROM seat_allocations WHERE booking_id = p_booking;
        UPDATE bookings SET status = 'expired' WHERE booking_id = p_booking;
        SET v_status = 'expired';
    END IF;
    COMMIT;
    SELECT v_status AS outcome;
END//

CREATE PROCEDURE confirm_payment(IN p_booking INT UNSIGNED,
    IN p_provider VARCHAR(50), IN p_event VARCHAR(191), IN p_amount DECIMAL(10,2))
SQL SECURITY DEFINER
BEGIN
    DECLARE v_status VARCHAR(20);
    DECLARE v_expiry DATETIME;
    DECLARE v_total DECIMAL(10,2);
    DECLARE v_event_booking INT UNSIGNED;
    DECLARE v_event_amount DECIMAL(10,2);
    DECLARE v_event_status VARCHAR(20);
    DECLARE v_event_type VARCHAR(30);
    DECLARE v_items INT;
    DECLARE v_allocations INT;
    DECLARE v_sum DECIMAL(10,2);
    DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;
    IF p_amount IS NULL OR p_amount < 0 OR p_provider IS NULL OR p_provider = ''
    OR p_event IS NULL OR p_event = '' THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'invalid payment';
    END IF;
    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
    START TRANSACTION;
    SELECT status, hold_expires_at, total_amount INTO v_status, v_expiry, v_total
    FROM bookings WHERE booking_id = p_booking FOR UPDATE;
    IF v_status IS NULL THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'unknown booking'; END IF;
    CALL lock_booking_inventory(p_booking);
    INSERT INTO payment_events (provider, provider_event_id, booking_id, event_type, amount)
    VALUES (p_provider, p_event, p_booking, 'payment_succeeded', p_amount)
    ON DUPLICATE KEY UPDATE payment_event_id = payment_event_id;
    SELECT booking_id, amount, processing_status, event_type

```

```

INTO v_event_booking, v_event_amount, v_event_status, v_event_type
FROM payment_events WHERE provider = p_provider AND provider_event_id = p_event FOR UPDATE;
IF v_event_booking <> p_booking OR v_event_amount <> p_amount OR v_event_type <>
'payment_succeeded' THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'event payload mismatch';
END IF;
IF v_event_status = 'pending' THEN
    SELECT COUNT(*), COALESCE(SUM(purchase_price), 0) INTO v_items, v_sum
    FROM booking_seats WHERE booking_id = p_booking;
    SELECT COUNT(*) INTO v_allocations FROM seat_allocations WHERE booking_id = p_booking;
    IF v_status = 'held' AND v_expiry > SYSDATE() AND v_total = p_amount
        AND v_sum = v_total AND v_items > 0 AND v_allocations = v_items
        AND NOT EXISTS (SELECT 1 FROM booking_seats bs LEFT JOIN seat_allocations a
            ON a.booking_id = bs.booking_id AND a.row_label = bs.row_label
            AND a.seat_number = bs.seat_number
            WHERE bs.booking_id = p_booking AND a.booking_id IS NULL) THEN
        UPDATE bookings SET status = 'confirmed' WHERE booking_id = p_booking;
        SET v_event_status = 'applied';
    ELSE
        IF v_status = 'held' AND v_expiry <= SYSDATE() THEN
            DELETE FROM seat_allocations WHERE booking_id = p_booking;
            UPDATE bookings SET status = 'expired' WHERE booking_id = p_booking;
        END IF;
        SET v_event_status = 'refund_required';
    END IF;
    UPDATE payment_events SET processing_status = v_event_status, processed_at = SYSDATE()
    WHERE provider = p_provider AND provider_event_id = p_event;
END IF;
COMMIT;
SELECT v_event_status AS outcome;
END//
DELIMITER ;

```

P2: theatre/date showtime query

```
-- P2: List shows for a selected theatre and date.
-- Batch 12: two statements (inputs and query). Read-only apart from session variables.
-- Fixed fixture date for repeatable verification; edit inputs to select another day.
SET @theatre_id = 1, @selected_date = DATE('2026-09-28');

SELECT m.title AS movie_title,
       sc.screen_name,
       DATE(sh.starts_at) AS show_date,
       TIME(sh.starts_at) AS show_time
FROM movie_seat_booking.shows AS sh
JOIN movie_seat_booking.movies AS m ON m.movie_id = sh.movie_id
JOIN movie_seat_booking.screens AS sc ON sc.screen_id = sh.screen_id
JOIN movie_seat_booking.theatres AS th ON th.theatre_id = sc.theatre_id
WHERE th.theatre_id = @theatre_id
      AND sh.starts_at >= @selected_date
      AND sh.starts_at < DATE_ADD(@selected_date, INTERVAL 1 DAY)
ORDER BY m.title, sh.starts_at, sh.show_id;
```